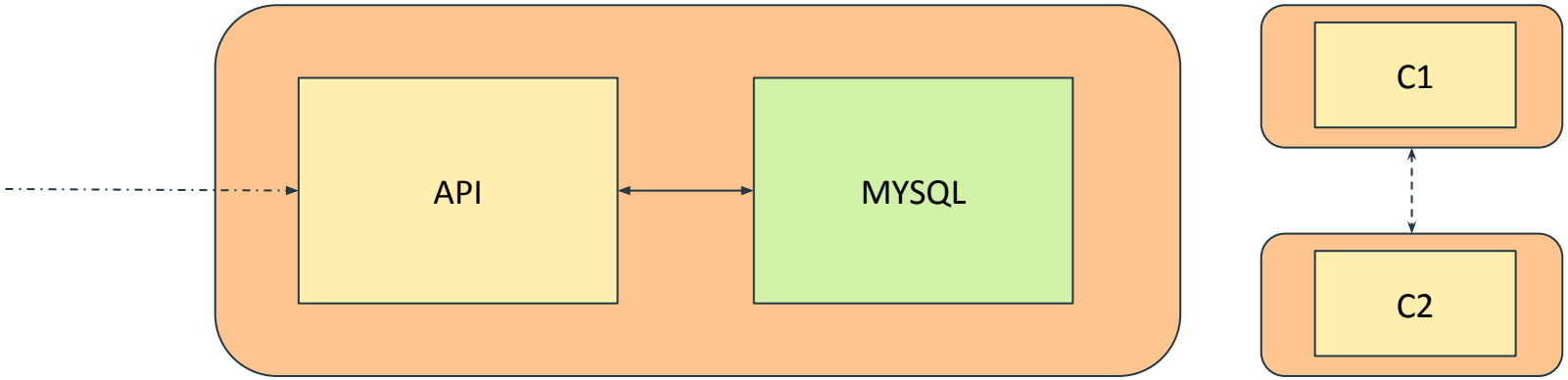
docker-compose
introduction et orchestration

A.SCHUTZ

docker-compose

Docker Compose est un outil de gestion d'orchestration de conteneurs qui utilise un fichier YAML (comme pour les workflows github actions) pour définir et exécuter des applications multi-conteneurs Docker. Il simplifie le processus de création, de démarrage et d'arrêt de services complexes en les définissant comme des services. Cela permet de gagner en productivité et de faciliter la collaboration entre développeurs."



Fichier .env et .env.example

Le fichier `.env` est un fichier de configuration qui stocke des variables d'environnement de manière sécurisée. Il permet de séparer les informations sensibles de votre code source, facilitant ainsi la gestion des différents environnements (développement, production, etc.).



Attention! le `.env` se met dans le `.gitignore`
Un `.env.example` est donné dans le dépôt avec les clés et sans les valeurs

```
uv add python-dotenv
```

```
MYSQL_ROOT_PASSWORD=your_strong_password
MYSQL_DATABASE=ma_base

DB_PASSWORD=mon_mot_de_passe_secret
DB_HOST=localhost
```

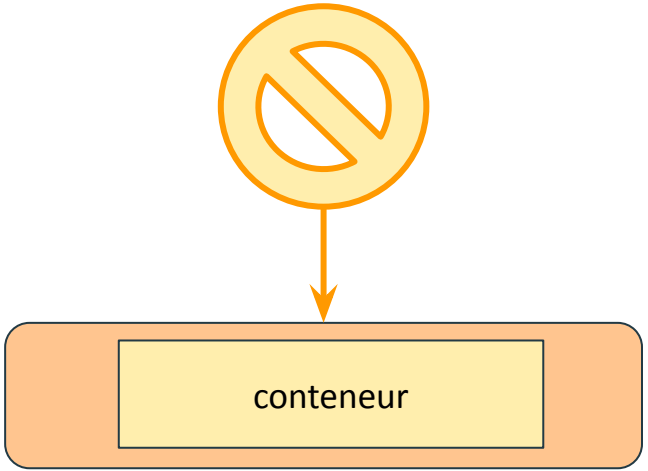
```
import os
from dotenv import load_dotenv
load_dotenv()

# Accéder aux variables
database url = os.getenv('DATABASE URL')
secret_key = os.getenv('SECRET_KEY')

print(database url)
print(secret_key)
```

Les variables d’environnement dans un docker compose, exemple

```yaml
services:
  db:
    image: mysql:latest
    ports:
      - "${DB_PORT:-3310}:3306"
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
```

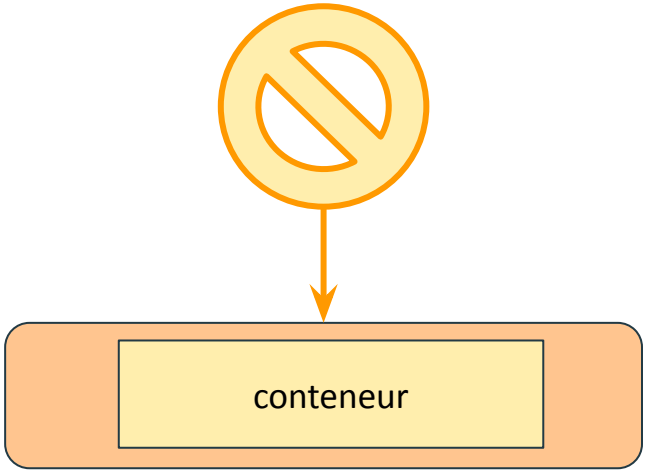


Les variables d’environnement dans un docker compose

Chaque service est un conteneur est les variables contenu dans **`environnement`** ne sont connues que par ce conteneur

La syntaxe pour récupérer les variables du **`.env`** est

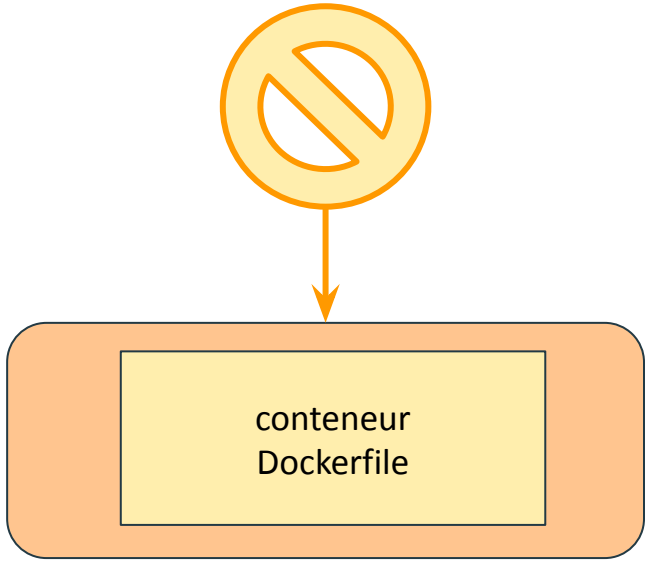
```
variable_du_conteneur=${variable}
```



Les variables d’environnement dans un docker compose et dockerfile

En terme de bonne pratique il vaut mieux transférer les variables d’**`environnements`** vers votre api ou application.

Cependant, et dans le cas d’une **`API`**, il faut garder en tête qu’à la fin c’est **`docker-compose`** qui fait le mapping finale de votre **`URL`** et de votre **`port`**.



commandes les plus courantes

`docker-compose up` : Démarre les services de votre application. C'est la commande la plus utilisée. Utilisez l'option `-d` pour lancer les conteneurs en arrière-plan (mode détaché).

`docker-compose down` : Arrête et supprime les conteneurs, les réseaux et les volumes créés par `up`.

`docker-compose build` : Construit ou reconstruit les images de vos services.

`docker-compose start` : Démarre les services qui sont déjà créés.

commandes les plus courantes

`docker-compose stop` : Arrête les services sans supprimer les conteneurs.

`docker-compose restart` : Redémarre les services.

`docker-compose logs` : Affiche les logs des conteneurs.

`docker-compose ps` : Affiche l'état des conteneurs (en cours d'exécution, arrêtés, etc.).

`docker-compose exec <service> <commande>` : Exécute une commande dans un conteneur en cours d'exécution.

`docker system prune` : RAZ

~~version~~

services

image

build

ports

volumes

environment

depends_on

networks

command

restart

```
networks:
  mon_reseau_overlay:
    driver: overlay
    attachable: true
```

Veille technologique individuelle 30 min

```
networks:
mon_reseau_prive:
  driver: bridge
```

```
ipam:
  config:
    - subnet: 172.20.0.0/16  # Réseau principal
      gateway: 172.20.0.1
    - subnet: 172.21.0.0/24  # Sous-réseau 1
    - subnet: 172.22.0.0/24  # Sous-réseau 2
```

30 MINUTES LATER

Fun pour les dev web

```yaml
networks:
  my_network:

services:
  apache:
    image: php:7.4-apache
    container_name: apache
    volumes:
      - ./www:/var/www/html
    ports:
      - "${APACHE_PORT}:80"
    environment:
      - APACHE_DOCUMENT_ROOT=/var/www/html
    depends_on:
      - mysql
    env_file:
      - .env
    networks:
      - my_network
```

```yaml
  mysql:
    image: mysql:5.7
    container_name: mysql
    environment:
      MYSQL_ROOT_PASSWORD: "${MYSQL_ROOT_PASSWORD}"
      MYSQL_DATABASE: "${MYSQL_DATABASE}"
    ports:
      - "${MYSQL_PORT}:3306"
    env_file:
      - .env
    networks:
      - my_network

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: phpmyadmin
    environment:
      - PMA_HOST=mysql
      - PMA_USER=root
      - PMA_PASSWORD=rootpassword
    ports:
      - "8081:80"
    depends_on:
      - mysql
    networks:
      - my_network
```

On joue avec quelques exemples

Plusieurs démos:

- Transfert des variables d'environnement vers une base de données.
- Transfert des variables d'environnement vers une application **`streamlit`**.
- Transfert des variables d'environnement vers une api **`Fast API`** et comprendre pourquoi le dockerfile est comme il est.
- Stack **`WAMP/MAMP/XAMP`** bien connue des développeurs web **`php.`**

`https://github.com/DeVerMyst/docker-compose-intro.git`

Limiter la RAM utilisé

```yaml
services:
  api:
    deploy:
      resources:
        limits:
          memory: 512M
  prometheus:
    image: prom/prometheus:latest
    deploy:
      resources:
        limits:
          memory: 512M
```

```yaml
api:
  build: ./fastapi_app
  ports:
    - "${FASTAPI_PORT}:8080"
  environment:
    DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
  deploy:
    resources:
      limits:
        memory: 512M # Limite pour l'API FastAPI
  depends_on:
    - db
  networks:
    - app_network
```

Orchestrer un projet complet

pour des questions de places je n’ai pas mis:

- .gitignore
- .github/workflows
- README.md

pourquoi il n’y a pas de .venv?
pourquoi il y a plusieurs pyproject.toml?

```
├── API_BDD
│   ├── Dockerfile
│   ├── api_bdd.py
│   ├── pyproject.toml
│   └── modules_BDD
│       ├── __init__.py
│       ├── create_tables.py
│       ├── filter_data.py
│       ├── models.py
│       └── scrapt_data.py
├── API_IA
│   ├── Dockerfile
│   ├── api_ia.py
│   ├── pyproject.toml
│   └── TOOLS_IA
│       ├── __init__.py
│       ├── benchmark_model.py
│       └── train_model.py
├── APP
│   ├── Dockerfile
│   ├── main.py
│   └── pyproject.toml
├── docker-compose.yml
└── .env
```

```yaml
version: "3.9"

services:
  api_bdd:
    build: ./API_BDD
    ports:
      - "8001:8001"
    depends_on:
      - db
    environment:
      DATABASE_URL: ${DATABASE_URL}
      # autres variables d'environnement

  api_ia:
    build: ./API_IA
    ports:
      - "8002:8002"
    depends_on:
      - db
    environment:
      DATABASE_URL: ${DATABASE_URL}
      # autres variables d'environnement
```

```yaml
  app:
    build: ./APP
    ports:
      - "8000:8000"
    depends_on:
      - api_bdd
      - api_ia
    environment:
      API_BDD_HOST: ${API_BDD_HOST}
      API_BDD_PORT: ${API_BDD_PORT}
      API_IA_HOST: ${API_IA_HOST}
      API_IA_PORT: ${API_IA_PORT}
      # autres variables d'environnement

  db:
    image: postgres:latest  # mysql:latest
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
```

où sont les erreurs?

```yaml
depends_on: [mother, database, redis]
```

sécurité des variables d'environnement

https://github.com/gitleaks/gitleaks

Gitleaks est un petit utilitaire qui permet de vérifier que vous ne faites pas de push de secret important afin d'augmenter la sécurité de votre repos GitHub

```yaml
name: Security Scan (Gitleaks)

on:
  push:
    branches: [ main, dev ]
  pull_request:
    branches: [ main ]

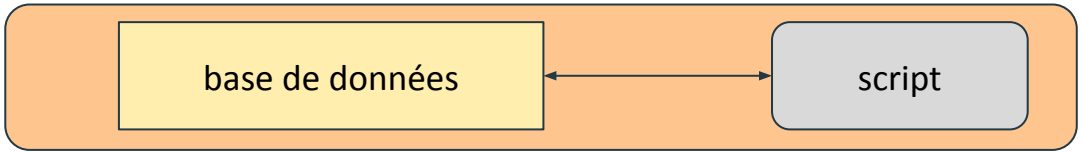
jobs:
  gitleaks:
    name: Gitleaks Secret Detection
    runs-on: ubuntu-latest
    steps:
      # fetch-depth: 0 récupère tout l'historique Git.
      # sinon Gitleaks ne voit que le dernier commit.
      - name: Checkout Code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run Gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        with:
          fail: true # Fait échouer la CI si un secret est trouvé
```

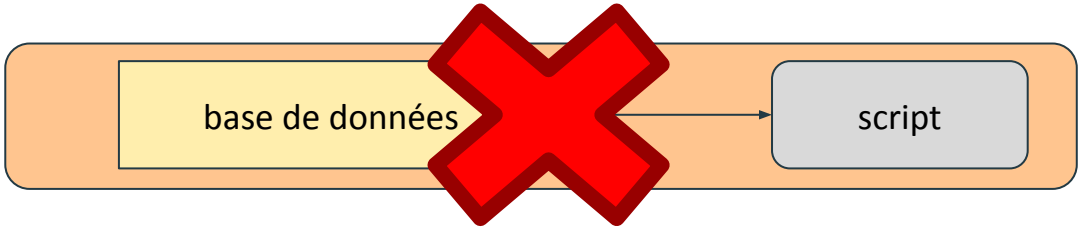
Volumes

Persistence des données

docker-compose up



docker-compose down

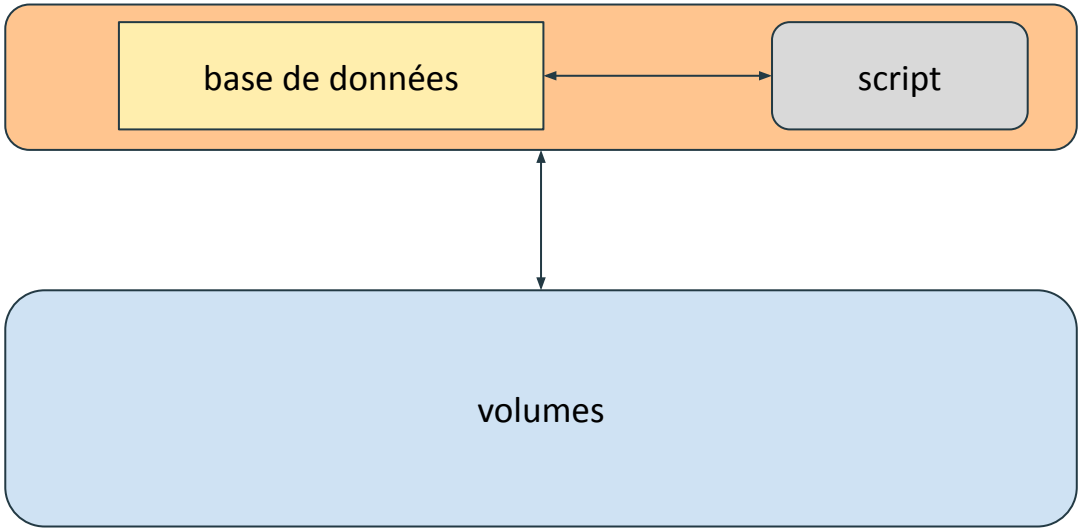


Ca peut être gênant de tout perdre quand on éteint le conteneur

Persistence des données

C’est l’intérêt d’utiliser des volumes car ils stockent les informations à un autre endroit.

- Contenu d’une base de données
- Rapport ML Flow
- modèle pré-entraîné



où sont les erreurs?

```yaml
version: '3.8'
services:
  db:
    image: postgres:13
    volumes:
      - db_data:/var/lib/postgresql/data # Monte le volume dans le répertoire de données de PostgreSQL
    environment:
      POSTGRES_USER: mon_utilisateur
      POSTGRES_PASSWORD: mon_mot_de_passe
      POSTGRES_DB: ma_base_de_donnees
    ports:
      - "5432:5432" # Mapping du port de la base de données vers l'hôte
```

Le volume nommé `db_data` est créé et monté dans le répertoire `/var/lib/postgresql/data` du conteneur PostgreSQL.

où sont les erreurs?

```yaml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      mlflow:
        condition: service_started
    environment:
      MLFLOW_TRACKING_URI: http://mlflow:5000
    restart: unless-stopped
    networks:
      - my_network
```

```yaml
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.9.0
    ports:
      - "5000:5000"
    volumes:
      - ./mlruns:/mlruns
      - ./artifacts:/artifacts

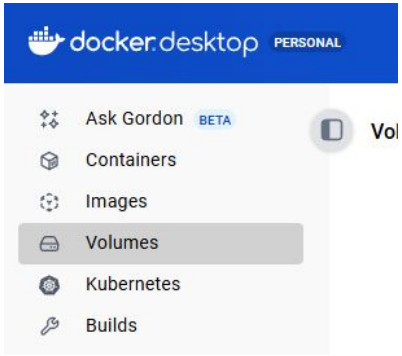
    environment:
      MLFLOW_TRACKING_URI: http://mlflow:5000
      MLFLOW_ARTIFACT_URI: /mlruns
    command: mlflow server xxxx xxxxx xxxxxx xxxxx
    restart: unless-stopped
    networks:
      - my_network



networks:
  my_network:
    driver: bridge
```

exemple d’une api avec monitoring ML Flow

Conteneur - Image - Volume



Conteneur c'est l'exécution d'une Image (une même image peut être lancé plein de fois et du coup on à plein de conteneur)

L'image est créé à partir du Dockerfile

Volume est un espace de stockage (virtuel si stocké par docker ou physique si configuré en local)

Pour un stockage virtuel:

```
nom_du_volume:/ chemin_fichier_sauvegardé
```

```
modeles:/app/modeles
```

Pour un stockage local:

```
chemin_local :/ chemin_fichier_sauvegardé
```

```
./modeles:/app/modeles
```

```yaml
version: '3.8'
services:
  mon_ia:
    build: .
# Construit l'image à partir du Dockerfile
    volumes:
      - ./modeles:/app/modeles
# Monte le répertoire ./modeles de l'hôte
      - ./donnees_entrainement:/app/donnees
```

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "mon_script.py"]
```

les modèles entraînés et sauvegardés

```python
import os
import tensorflow as tf # Exemple avec Tensorflow

# Chemin vers les modèles montés
model_path = "/app/modeles/mon_modele.h5"
# Chemin vers les données d'entraînement montées
data_path = "/app/donnees/data.csv"

# Vérifie si le fichier de modèle existe
if os.path.exists(model_path):
    model = tf.keras.models.load_model(model_path)
else:
    print(f"Modèle non trouvé à : {model_path}")

# Vérifie si le fichier de données existe
if os.path.exists(data_path):
    print(f"Données trouvées : {data_path}")
else:
    print(f"Données non trouvées à : {data_path}")

# sauvegarde dans le volume de modeles
with open("/app/modeles/test.txt", "w") as f:
    f.write("test")
```

On joue un peu avec les volumes

https://github.com/DeVerMyst/docker-compose-volumes.git

Réseaux Docker

Réseaux docker

Dans Docker Compose, les réseaux jouent un rôle clé pour organiser et sécuriser la communication entre vos services.

Les réseaux personnalisés permettent d'isoler des groupes de conteneurs, améliorant ainsi la sécurité et la performance.

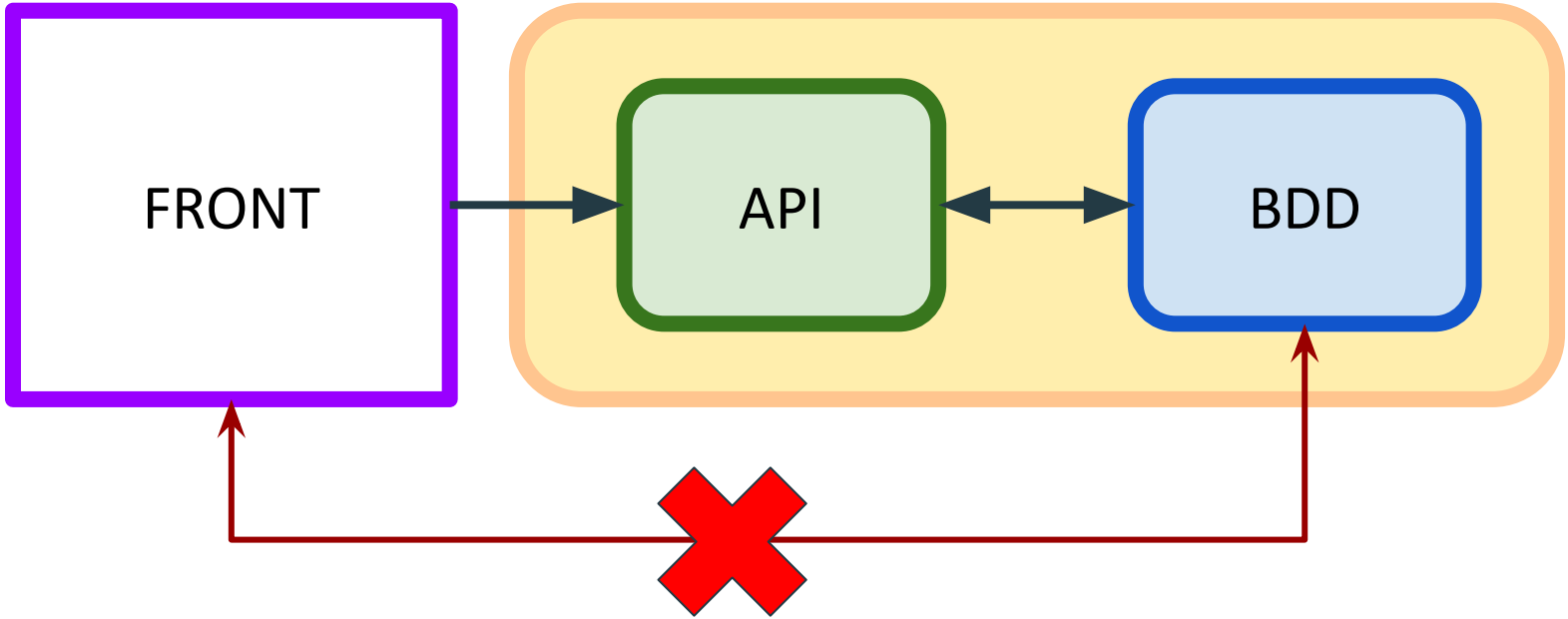
Imaginez une application d'IA qui a besoin d'une base de données pour stocker les résultats de ses analyses.

Quel est la différence avec une API backend qui communique avec une bdd directement

Sans réseaux personnalisés : Si vous avez une API et une base de données dans votre `docker-compose.yml`, l'API peut accéder à la base de données mais potentiellement n'importe quel autre service pourrait également accéder à la base de données.

Avec réseaux personnalisés : Vous créez un réseau "backend" et connectez l'API et la base de données à ce réseau. Seul l'API pourra donc communiquer avec la base de données. La sécuritée est donc améliorée.

réseau privé

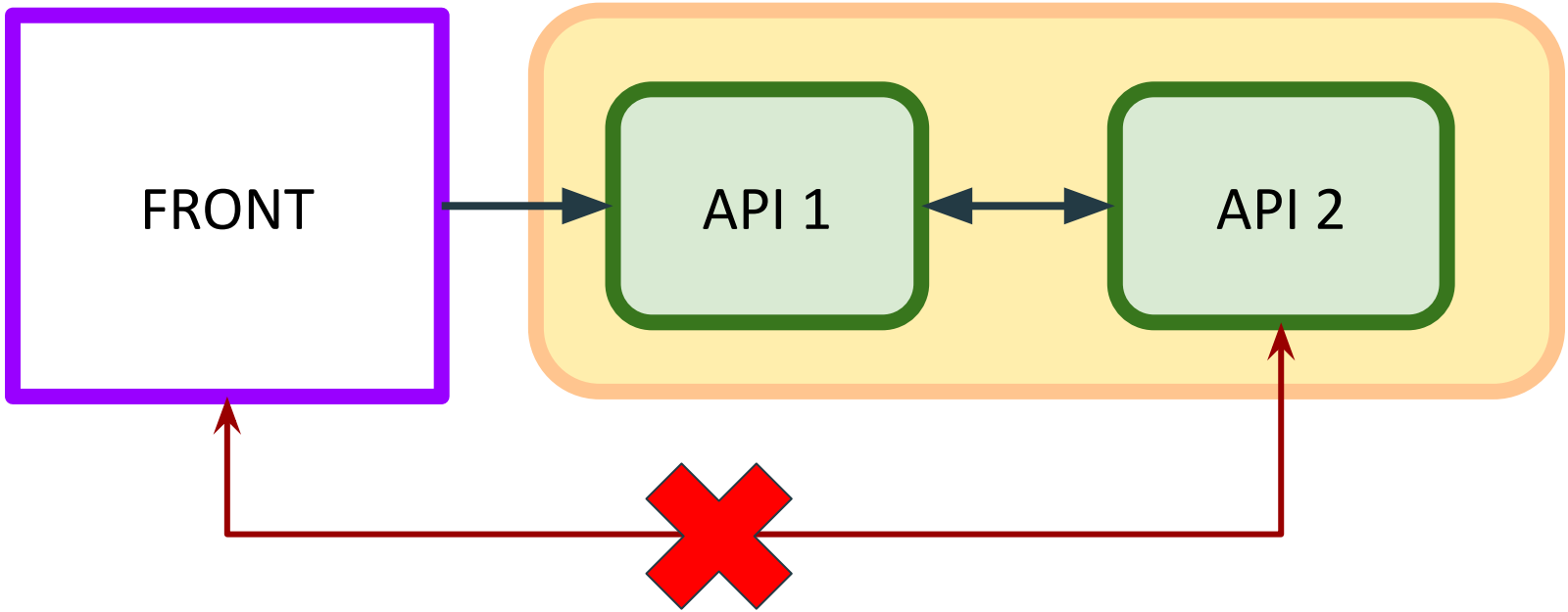


```yaml
version: '3.8'
services:
  mother:
    build:
      context: ./mother_api
    ports:
      - "8000:8000"
    networks:
      - app_network
  child:
    build:
      context: ./child_api
    depends_on: [mother]
    networks:
      - app_network
networks:
  app_network:
```

```
CHILD_API_URL = "http://child:8001"
```

```
├── mother api/
│   ├── mother_api.py
│   ├── pyproject.toml
│   └── Dockerfile
├── child_api/
│   ├── child api.py
│   ├── pyproject.toml
│   └── Dockerfile
├── .venv
├── .gitignore
├── .dockerignore
├── README.md
└── docker-compose.yml
```

réseau privé



On joue un peu avec le réseau

https://github.com/DeVerMyst/docker-compose-networks.git